

Algorithms

Lecture #13

Syed Hamed Raza

Quick sort

Quicksort

- Our next sorting algorithm is Quicksort.
- It is one of the fastest sorting algorithms known and is the method of choice in most sorting libraries.
- Quicksort is based on the divide and conquer strategy.

Quick sort

Quicksort

```
QUICKSORT( array A, int p, int r)
1  if (r > p)
2    then
3      i ← a random index from [p..r]
4      swap A[i] with A[p]
5      q ← PARTITION(A, p, r)
6      QUICKSORT(A, p, q - 1)
7      QUICKSORT(A, q + 1, r)
```

Partition algorithm

Partition Algorithm

Recall that the partition algorithm partitions the array $A[p..r]$ into three sub arrays about a pivot element x .

- $A[p..q - 1]$ whose elements are less than or equal to x ,
- $A[q] = x$,
- $A[q + 1..r]$ whose elements are greater than x

Choosing the Pivot

Choosing the Pivot

- We will choose the first element of the array as the pivot, i.e. $x = A[p]$.
- If a different rule is used for selecting the pivot, we can swap the chosen element with the first element.
- We will choose the pivot randomly.

Partition algorithm

Partition Algorithm

The algorithm works by maintaining the following *invariant condition*.

1. $A[p] = x$ is the pivot value.
2. $A[p..q - 1]$ contains elements that are less than x .
3. $A[q + 1..s - 1]$ contains elements that are greater than or equal to x
4. $A[s..r]$ contains elements whose values are currently unknown.

Partition algorithm

Partition Algorithm

```
PARTITION( array A, int p, int r)
1  x ← A[p]
2  q ← p
3  for s ← p + 1 to r
4  do if (A[s] < x)
5      then q ← q + 1
6          swap A[q] with A[s]
7
8  swap A[p] with A[q]
9  return q
```

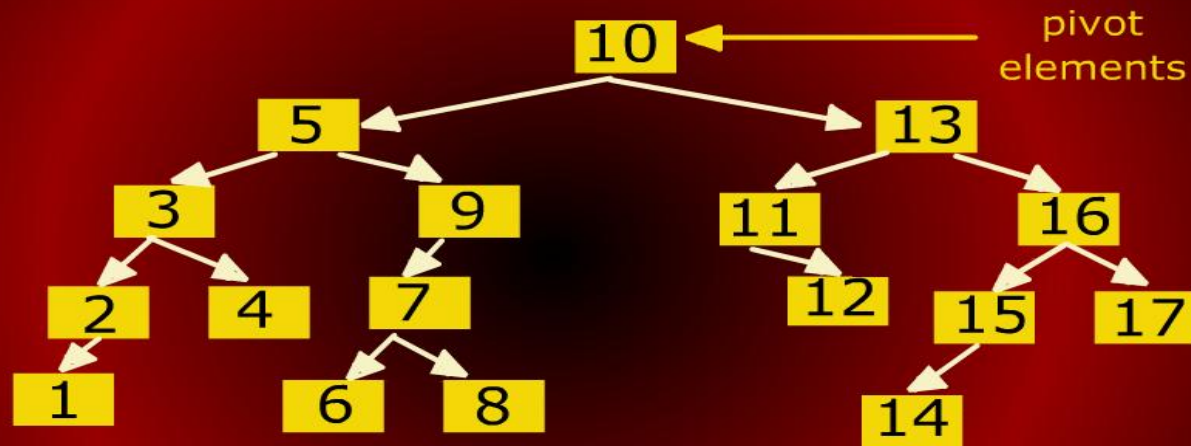
Partition algorithm

Partition Algorithm



Quick Sort BST

Quicksort BST



Analysis of Quick sort

Analysis of Quicksort

- The running time of quicksort depends heavily on the selection of the pivot.
- If the rank of the pivot is very large or very small then the partition (BST) will be unbalanced.
- Since the pivot is chosen randomly in our algorithm, the expected running time is $O(n \log n)$.

Analysis of Quick sort

Analysis of Quicksort

- It takes $\Theta(n)$ time to do the partitioning.
- The time taken for sorting array $A[1..n]$ is
$$T(n) = T(q - 1) + T(n - q) + n$$
- To get the worst case, we maximize over all possible values of q .

Analysis of Quick sort

Analysis of Quicksort

Putting is together, we get the recurrence

$$T(n) = \begin{cases} 1 & \text{if } n \leq 1 \\ \max_{1 \leq q \leq n} (T(q-1) + T(n-q) + n) & \text{otherwise} \end{cases}$$

The worse case happens at $q = 1$ or $q = n$. If we expand the recurrence for $q = 1$, we get:

Analysis of Quick sort

Analysis of Quicksort

$$\begin{aligned}T(n) &\leq T(0) + T(n-1) + n \\&= 1 + T(n-1) + n \\&= T(n-1) + (n+1) \\&= T(n-2) + n + (n+1) \\&= T(n-3) + (n-1) + n + (n+1) \\&= T(n-4) + (n-2) + (n-1) + n \\&\quad + (n+1) \\&= T(n-k) + \sum_{i=-1}^{k-2} (n-i)\end{aligned}$$

Analysis of Quick sort

Analysis of Quicksort

For the basis $T(1) = 1$ we set
 $k = n - 1$ and get

$$\begin{aligned} T(n) &\leq T(1) + \sum_{i=-1}^{n-3} (n - i) \\ &= 1 + (3 + 4 + 5 + \dots \\ &\quad + (n - 1) + n + (n + 1)) \\ &\leq \sum_{i=1}^{n+1} i = \frac{(n + 1)(n + 2)}{2} \in \Theta(n^2) \end{aligned}$$

Average Case Analysis of Quick sort

Average-case Analysis of Quicksort

- We will now show that in the average case, quicksort runs in $\Theta(n \log n)$ time.
- Recall that when we talked about average case at the beginning of the semester, we said that it depends on some assumption about the distribution of inputs.

Quick sort Trace

Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
7	6	4	3	9	8	2	1	5	10	17	15	16	14	11	12	13
1	2	4	3	5	8	6	7	9	10	12	11	13	14	15	17	16
1	2	3	4	5	8	6	7	9	10	11	12	13	14	15	16	17
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

Analysis of Quicksort

Analysis of Quicksort

- If the rank of the pivot is very large or very small then the partition (BST) will be unbalanced.
- Since the pivot is chosen randomly in our algorithm, the expected running time is $O(n \log n)$.
- The worst case time, however, is $O(n^2)$. Luckily, this happens rarely.

Analysis of Quick sort worse case

Design and Analysis of Algorithms

Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
---	---	----	---	----	---	---	---	----	----	----	---	----	----	---	---	----

Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
7	6	4	3	9	8	2	1	5	10	17	15	16	14	11	12	13

Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
7	6	4	3	9	8	2	1	5	10	17	15	16	14	11	12	13

Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
7	6	4	3	9	8	2	1	5	10	17	15	16	14	11	12	13
1	2	4	3	5	8	6	7	9	10	12	11	13	14	15	17	16

Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
7	6	4	3	9	8	2	1	5	10	17	15	16	14	11	12	13
1	2	4	3	5	8	6	7	9	10	12	11	13	14	15	17	16

Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
7	6	4	3	9	8	2	1	5	10	17	15	16	14	11	12	13
1	2	4	3	5	8	6	7	9	10	12	11	13	14	15	17	16
1	2	3	4	5	8	6	7	9	10	11	12	13	14	15	16	17

Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
7	6	4	3	9	8	2	1	5	10	17	15	16	14	11	12	13
1	2	4	3	5	8	6	7	9	10	12	11	13	14	15	17	16
1	2	3	4	5	8	6	7	9	10	11	12	13	14	15	16	17

Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
7	6	4	3	9	8	2	1	5	10	17	15	16	14	11	12	13
1	2	4	3	5	8	6	7	9	10	12	11	13	14	15	17	16
1	2	3	4	5	8	6	7	9	10	11	12	13	14	15	16	17
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

Quicksort Trace

7	6	12	3	11	8	7	1	15	13	17	5	16	14	9	4	10
7	6	4	3	9	8	2	1	5	10	17	15	16	14	11	12	13
1	2	4	3	5	8	6	7	9	10	12	11	13	14	15	17	16
1	2	3	4	5	8	6	7	9	10	11	12	13	14	15	16	17
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

Analysis of Quicksort

Average case